



QVM: Quantum Gate Virtualization Machine

NATHANIEL TORNOW, TU Munich, Germany and Leibniz Supercomputing Centre, Germany

EMMANOUIL GIORTAMIS, TU Munich, Germany

PRAMOD BHATOTIA, TU Munich, Germany

We present the Quantum Gate Virtualization Machine (QVM), an end-to-end generic system for scalable execution of large quantum circuits with high fidelity on noisy and small quantum processors (QPUs) by leveraging gate virtualization. QVM exposes a virtual circuit intermediate representation (IR) that extends the notion of quantum circuits to incorporate gate virtualization. Based on the virtual circuit as our IR, we propose the QVM compiler—an extensible compiler infrastructure to transpile a virtual circuit through a series of modular optimization passes to produce a set of optimized circuit fragments. Lastly, these transpiled circuit fragments are executed on QPUs using our QVM runtime—a scalable and parallel infrastructure to virtualize and execute circuit fragments on a set of QPUs.

We evaluate QVM on IBM’s 7- and 27-qubit QPUs. Our evaluation shows that our approach allows for the execution of circuits with up to double the number of qubits compared to the qubit-count of a QPU, while improving fidelity by 4.7× on average compared to larger QPUs and that we can effectively reduce circuit depths to only 40% of the original circuit depths.

CCS Concepts: • **Computer systems organization** → **Quantum computing**.

Additional Key Words and Phrases: Quantum Circuit Optimization and Compilation, Circuit Cutting

ACM Reference Format:

Nathaniel Tornow, Emmanouil Giortamis, and Pramod Bhatotia. 2025. QVM: Quantum Gate Virtualization Machine. *Proc. ACM Program. Lang.* 9, PLDI, Article 187 (June 2025), 26 pages. <https://doi.org/10.1145/3729290>

1 Introduction

Quantum computers promise to solve otherwise intractable problems in optimization [22], factorization [78], or quantum simulation [36, 66]. However, the reliable operation of quantum processing units (QPUs) is extremely challenging, as the same properties that could lead to computational benefits are also the main reason for uncontrollable noise and state-decoherence during a quantum computation on a QPU[69]. This still severely limits the number of qubits and operations we can run within the same quantum program.

Gate virtualization (GV) has recently been proposed to scale the size of quantum programs running with high fidelity on small and noisy QPUs [47]. This technique virtualizes two-qubit qubit gates by executing a predefined set of single-qubit operations instead, and reconstructs the result of the original circuit via classical postprocessing. Theoretical work shows that GV allows quantum circuits to be optimized to scale and improve fidelity in two different dimensions: First, quantum circuits can be decomposed into multiple smaller circuit fragments to run on small QPUs [47, 65, 67], and second, circuit depth can be reduced to increase overall fidelity [8, 95].

However, the effectiveness of gate virtualization is severely hampered by the lack of general and extensible procedures for automatically applying and executing gate virtualization. Previous

Authors’ Contact Information: [Nathaniel Tornow](#), TU Munich, Munich, Germany and Leibniz Supercomputing Centre, Munich, Germany, nathaniel.tornow@tum.de; [Emmanouil Giortamis](#), TU Munich, Munich, Germany, emmanouil.giortamis@tum.de; [Pramod Bhatotia](#), TU Munich, Munich, Germany, pramod.bhatotia@tum.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/6-ART187

<https://doi.org/10.1145/3729290>

studies have primarily concentrated on utilizing gate virtualization through ad-hoc methods or on an individual application level [7, 8, 95]. Moreover, the applicability of gate virtualization suffers greatly from the high computational cost, since virtualizing k two-qubit gates comes with a quantum circuit and classical post-processing overhead of $O(6^k)$ [47].

To this end, we target the following research question: *How can we design a generic and extensible system that fully utilizes the full potential of GV to scale the size of circuits that can be executed with high fidelity on current QPUs, despite the computational overhead?*

To address this research question, we introduce the Quantum Gate Virtualization Machine (QVM), a system for scalable and reliable execution of quantum circuits on small and noisy QPUs by fully leveraging GV. QVM therefore simulates a larger, noise-mitigated QPU, which internally comprises multiple smaller, noisy QPUs. We make the following key contributions:

- To enable the general and programmable application of GV, and as the basis of our work, we present the **virtual circuit IR** (VC-IR). The VC-IR extends the quantum circuit abstraction, manages virtual gates, decompositions into several smaller circuit fragments, and data structures to efficiently analyze and apply GV (§ 4.2).
- To enable automatic and efficient GV that optimizes a circuit with as little post-processing as possible, we introduce the **QVM Compiler**, an extensible pipeline for converting large quantum circuits into optimized virtual circuits (§ 4). The compiler enables multiple optimization passes that apply GVs based on the VC-IR. We present three generic optimization passes for efficient gate virtualization on arbitrary quantum circuits. (1) The *circuit cutter* (§ 4.3) decomposes a circuit into several smaller fragments to run on smaller QPUs, (2) the *dependency reducer* (§ 4.4) reduces the dependencies within a circuit to reduce error propagation and the number of SWAP gates, and (3) the *qubit reuser* (§ 4.5) applies a qubit reuse technique to enable a trade-off between overhead and circuit depth.
- To enable as many GVs as possible to benefit from its opportunities, we present the **QVM Runtime**, a scalable system that can run a virtual circuit (VC) on a set of QPUs and classical nodes (§ 5). The runtime uses the core component of a *virtualizer* (§ 5.2) that instantiates fragments of the VC and computes the result of the VC using highly parallel post-processing. The quantum circuits are executed scalably on QPUs using QVM's *QPU manager* (§ 5.3).

By fully incorporating GV, we establish a hybrid approach to solving complex problems that neither quantum nor classical computers can tackle alone. This enables the execution of larger quantum circuits and broadens the practical applications of quantum computation.

We implement QVM in Python, building on Qiskit [70], maintaining full hardware agnosticism. For our compiler, in addition to heuristic algorithms, we implement optimal passes using Answer Set Programming following our optimization models [24].

We evaluate QVM on IBM's 7- and 27-qubit QPUs and simulators using various circuits used in popular quantum algorithms [40, 71, 88]. Our analysis on real QPUs shows that we can execute circuits with up to $2\times$ the qubit count of the available QPUs while improving fidelity by an average of $4.7\times$ and up to $33.6\times$ (§ 6.1). Our intra-circuit dependency reduction techniques reduce the depth of transpiled circuits on average to 64% of the original circuit and increase fidelity by an average of $1.4\times$ and up to $5.2\times$ (§ 6.2). Our dependency reducer also enables the reuse of more qubits to reduce the width of circuits with less virtualization overhead (§ 6.3). Our QVM runtime scales efficiently and enables the execution of large virtual circuits that surpass current hardware limitations by several orders of magnitude with low memory requirements (§ 6.4-6.5).

2 Background and Motivation

2.1 Quantum Computations

We define quantum computation following the computational model introduced by Peng et al. [65]. A quantum computation consists of a quantum circuit acting on m qubits, initialized in the computational state $|0\rangle^{\otimes m}$, and measuring the expectation value of an observable O , denoted as $\langle O \rangle$ (Fig. 1, (a)). Our work focuses on computing the probabilities of measurement outcomes for a given quantum circuit, which corresponds to evaluating $\langle P_x \rangle$, the expectation value of the projection operator $P_x = |x\rangle\langle x|$ that projects onto the measurement outcome x , represented as a bitstring. In practice, this is achieved by sampling the a given circuit n times and estimating $\langle P_x \rangle$ as $\langle P_x \rangle \approx \frac{n_x}{n}$, where n_x denotes the number of times the outcome x is observed.

2.2 Impact of Circuit Properties in the NISQ Era

NISQ QPUs face challenges such as significant noise, limited qubit connectivity, and decoherence within microseconds [2, 33]. Additionally, imperfections in quantum gates and measurement operations introduce computational errors during program execution. For instance, two-qubit operations on superconducting QPUs have an error rate in the orders of 0.1% to 1%, varying between different hardware-vendors and implementation of qubits [2, 33]. For other QPU-types, such as neutral-atoms or ion-traps, two-qubit gate errors are in a similar range [34, 38].

Further, limited qubit connectivity necessitates non-local SWAP operations, each introducing three more CNOT operations [41]. Finally, qubit dependencies [31] amplify noise-propagation between operations and qubits [28], and restrict circuit placement on a QPU, leading to more SWAP operations. Systems that optimize circuits by reducing their depth, number of CNOT operations, and qubit dependencies, are essential for practical quantum computing.

2.3 Foundations of Gate Virtualization

Gate virtualization (GV) allows us to decompose two-qubit gates into a combination of single-qubit operations [47]. GV can be realized through the technique of Quasiprobability Decomposition.

Quasiprobability Decomposition (QPD). Quasiprobability Decomposition (QPD) is a powerful technique for implementing otherwise infeasible quantum operations on a QPU [47]. The key idea is to represent a desired quantum channel $\mathcal{F}$, which acts on a quantum state ρ and cannot be directly realized, as a quasi-probabilistic mixture of implementable channels $\mathcal{E}_i$ on a quantum computer:

$$\mathcal{F}(\rho) = \sum_i c_i \mathcal{E}_i(\rho),$$

where c_i are real-numbered coefficients. When computing the expectation value of an observable O , given by $\langle O \rangle = \text{Tr}[O\mathcal{F}(\rho)]$, we can express it as: $\langle O \rangle = \text{Tr}[O\mathcal{F}(\rho)] = \sum_i c_i \text{Tr}[O\mathcal{E}_i(\rho)] = \sum_i c_i \langle O \rangle_i$, where we define $\langle O \rangle_i$ as the expectation value of the observable O for the i th channel.

In the context of GV, we apply QPD to a unitary channel $\mathcal{U}(\rho) = U\rho U^\dagger$, where U represents a two-qubit gate. Thus, to virtualize a two-qubit gate U , we express it as a mixture of single-qubit channels, which, on a quantum computer, consists of single-qubit gates and projective measurements. Such decompositions have been demonstrated for widely used two-qubit gates, such as the CZ and RZZ gate families, each requiring a decomposition into a set of six different gates [47]. Notably, this approach generalizes to any two-qubit unitary U by finding a decomposition into appropriate channels $\mathcal{E}_i$ that can be implemented on a QPU [48, 76, 97]. This makes the circuit-level abstraction for QPD broadly hardware-independent and applicable to any circuit.

Gate virtualization. Based on the established theory on QPD, we show GV schematically in Fig. 1 (b). Instead of executing the original circuit's two-qubit gate, we can use QPD to instead

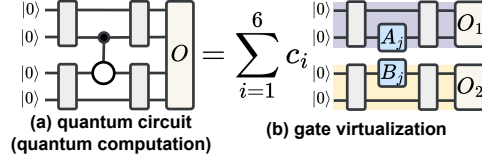


Fig. 1. Computational model and gate virtualization (§ 2.3). (a) A quantum computation to estimate the expectation value of an observable O . (b) Virtualizing a two-qubit gate by computing a weighted sum over circuit instances with single-qubit gates inserted ($O = O_1 \otimes O_2$)

calculate a weighted sum of six circuit instances to estimate $\langle O \rangle$. In each circuit instance i , instead of the original two-qubit gate, we insert A_i and B_i , which are either one-qubit unitary gates or projective measurements. This allows us to decompose each instance i into two smaller, completely independent sub-circuits, which can be sampled independently. We reconstruct the result with

$$\langle O \rangle = \sum_{i=1}^6 c_i \langle O \rangle_i = \sum_{i=1}^6 c_i \langle O_1 \rangle_i \langle O_2 \rangle_i, \quad (1)$$

where $\langle O \rangle_i$ is the result of each instance i and $O = O_1 \otimes O_2$. The property $O = O_1 \otimes O_2$ must hold to ensure that instances are decomposable into individual subcircuits. However, this is not a limitation, as every observable can be expressed as a linear combination of decomposable observables [55]. Furthermore, projection operators, which are the focus of this work, are also decomposable.

Virtualizing multiple gates. We now generalize GV to be applied on k gates in a circuit. We can think of adding a GV of another gate in a circuit as performing an additional GV for each instance of the original virtual gate.

To formalize the QPD of multiple gates in a circuit, let G_v be the set of all virtual gates in a quantum circuit. We then define a coefficient vector for each virtual gate $g \in G_v$ as $\mathbf{c}_g = (c_1, \dots, c_6)$. We define the global coefficient vector as $\mathbf{C} = \bigotimes_{g \in G_v} \mathbf{c}_g$, i.e., the tensor product of all individual coefficient vectors $\mathbf{c}_g$. Therefore, $\mathbf{C}$ is a vector with $|\mathbf{C}| = 6^k$ entries. To reconstruct the final results, we calculate

$$\langle O \rangle = \sum_{\mathbf{c}_i \in \mathbf{C}} c_i \prod_{j=1}^f \langle O_j \rangle_i, \quad (2)$$

which is the generalization of Eq. (1). Here f is the number of subcircuits, and $\langle O_j \rangle_i$ is the result of the j th subcircuit in the i th global instance. We must, therefore, calculate a sum over $|\mathbf{C}| = 6^k$ elements. Since in general $|\mathbf{C}| \gg f > |G_v|$, we need $\mathcal{O}(6^k)$ operations to calculate Eq. (2).

GV thus causes an exponential post-processing overhead of $\mathcal{O}(6^k)$ [47]. This severely limits the number of gates that can be virtualized within a circuit, meaning that we need to find a good compromise between the additional runtime and the benefits of GV, as described in the next section.

2.4 Motivation: Opportunities of Gate Virtualization

State-of-the-art circuit transpilers mainly focus on minimizing the circuit's post-transpilation depth and number of CNOTs. GV can be used effectively to reduce the width and qubit dependencies in a circuit, leading to improved execution fidelity for larger circuits on small QPUs, mostly at the cost of computational overheads, as shown in previous ad-hoc and theoretical work [47, 65, 75, 95]. In total, GV gives us opportunities in the two dimensions: In total, GV provides opportunities in two key dimensions: (1) reducing qubit *connectivity* (by cutting circuits) and (2) reducing qubit *dependency*. Here, we define two qubits as connected if they belong to the same (sub-)set of connected qubits (cf. Fig. 2 (b)). A qubit q_j is defined as dependent on another qubit q_i if an operation on q_j influences an operation on q_i [35] (cf. Fig. 2 (c)).

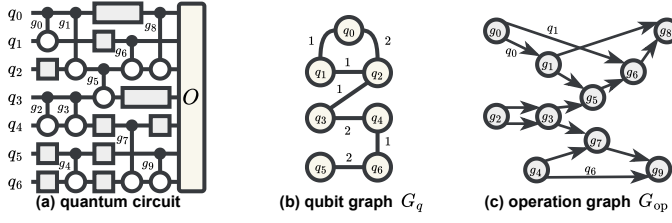


Fig. 2. Quantum circuit dependency structures (§ 3.1). (a) A quantum circuit, (b) the corresponding qubit graph G_q , modeling qubit connectivity, and (c) the operation graph G_{op} , modeling qubit dependencies.

Cutting quantum circuits. By virtualizing two-qubit gates, a circuit can be divided into smaller subcircuits, each with a lower number of qubits which can be run independently on small and noisy QPUs [47, 65], effectively reducing the qubit connectivity of the circuit. Circuits with lower widths exhibit less qubit mapping and routing constraints, therefore less post-transpilation CNOT operations and lower depth [53]. Moreover, as recent work shows, the wire-cutting technique [65, 83] can also be modeled using gate virtualization [14]. Therefore, a system that is able to virtualize gates is a complete solution for circuit cutting and knitting.

Reducing qubit dependencies. Gate virtualization can be used to virtualize gates that cause qubit dependencies. Virtualizing them and therefore reducing qubit dependencies in a circuit has three-fold advantages: Firstly, by reducing the propagation of errors through gates and qubits, fidelity can be improved. Secondly, reducing intra-circuit dependencies facilitates optimized qubit mapping and routing on QPUs during the transpilation process, which leads to lower depth and number of CNOTs. Lastly, fewer qubit dependencies enable the application of qubit-reuse [31, 35], which could enable a computationally efficient way to reduce circuit width.

To summarize, GV is a promising technique for improving the execution accuracy and scaling of quantum circuits, as shown in small ad-hoc examples in previous work. However, to fully exploit the benefits of GV on arbitrary circuits despite the exponential post-processing overhead, we need an automatic and efficient application as well as a scalable execution of GV.

3 Overview

We aim to design a system that can scale the sizes of circuits and practically execute them on QPUs with high fidelity. To realize this goal, gate virtualization (GV) is a promising technique that decomposes circuits into smaller circuit fragments or reduces the intra-dependencies in the circuit. However, dealing with the programming and exponential computational complexity of GV is challenging. We first describe the problem statement (§ 3.1), and next discuss these challenges and present our key ideas for addressing them (§ 3.2).

3.1 Problem Statement

We now formally describe the problems that arise from applying gate virtualization efficiently to (1) cut circuits into smaller subcircuits and (2) reduce dependencies between operations or qubits. **The circuit cutting problem.** To define the problem of optimal gate virtualization, we use the qubit graph G_q , which expresses the connection of qubits via two-qubit gates in a given circuit. The qubit graph $G_q = (V, E, w)$ is defined with the vertices $V = \{q_0, q_1, \dots\}$ representing the qubits of the circuit, and the edges $(q_i, q_j) \in E$ describe the connection between two qubits, which is the case when two a two-qubit gate exists between q_i and q_j . Each edge is weighted via $w : E \rightarrow \mathbb{N}$, where $w(q, q_j)$ indicates the number of two-qubit operations between the two qubits. We show an example of a qubit graph in Figure 2 (b).

Cutting a circuit into multiple subcircuits now corresponds to the following problem defined on the qubit graph: We need to partition G_q into k subgraphs $G_q^l = (V_l, E_l, w)$ with $V_l \subseteq V$ and $E_l \subseteq E$, where the number of vertices/ qubits is defined as $|V_l|$. The cut edges between the subgraphs are then $E_{\text{cut}} = \{(q_i, q_j) \in E \mid q_i \in V_a, q_j \in V_b, a \neq b\}$. The cut-weight, corresponding to the number of virtualized gates in the given graph-partition setting, corresponds to $n_{\text{cut}} = \sum_{(q_i, q_j) \in E_{\text{cut}}} w(q_i, q_j)$. To obtain an optimal cut, we need to solve the following optimization problem:

$$\begin{aligned} & \text{Minimize} && \sum_{(q_i, q_j) \in E_{\text{cut}}} w(q_i, q_j) + \alpha \cdot \frac{1}{k} \sum_{i=1}^k |V_i| \\ & \text{subject to} && |V_i| \leq s, \quad \forall i = 1, 2, \dots, k, \quad \bigcup_{i=1}^k V_i = V, \quad V_i \cap V_j = \emptyset, \quad \forall i \neq j \end{aligned} \quad (3)$$

Here, s is a constraint of the maximal number of vertices a subgraph can have, which corresponds to the maximal number of qubits a subcircuit is allowed to have, e.g., constraint by the maximum size of the available QPUs. We minimize the total number of gate virtualizations by the first term in the optimization while also minimizing the average size of the subcircuits to steer the optimization into the direction of minimizing the subcircuit sizes to generate less noise. The user can set the weighing factor α to prioritize minimizing weight cuts or the subcircuit sizes. We show an example of a operation graph in Figure 2 (c).

The dependency reduction problem. To define the dependency reduction problem, we use the operation graph G_{op} . The operation graph $G_{\text{op}} = (V, E, \phi)$ is directed acyclic graph (DAG), where the vertices $V = \{g_0, g_1, \dots\}$ are the operations of the circuit, and an edge $(g_i, g_j) \in E$ exists if the operation g_j directly follows g_i on a qubit in the circuit. Each vertex has assigned a set of qubits via $\phi : V \rightarrow \mathcal{P}(\{q_0, q_1, \dots\})$ that the respective operation acts on.

Now, we define a **operation dependency** $g_i \mapsto g_j$ when there exists a path from g_j to g_i in G_{op} . A **qubit dependency** $q_i \mapsto q_j$ exists, when there is a gate-dependency $g_k \mapsto g_l$, and $q_i \in \phi(g_k) \wedge q_j \in \phi(g_l)$. Let further $D(G_{\text{op}}) = \{(q_i, q_j) \mid q_i \mapsto q_j\}$ be the set of all qubit dependencies in G_{op} .

Our goal is now to minimize the number of qubit dependencies by virtualizing a set of gates $V_{\text{virt}} \subseteq V$. For this, let $G'_{\text{op}} = (V', G', \phi)$ the DAG when virtualizing the gates $g_i \in V_{\text{virt}}$, with $V' = V \setminus V_{\text{virt}}$. Let $G'_{\text{op}} = G_{\text{op}}[V_{\text{virt}}]$ the graph with the gates $g_i \in V_{\text{virt}}$ virtualized. We now want to optimize the following:

$$\begin{aligned} & \min_{V_{\text{virt}} \subseteq V} && |D(G_{\text{op}}[V_{\text{virt}}])| + |V_{\text{virt}}| \\ & \text{subject to} && |V_{\text{virt}}| \leq b \end{aligned} \quad (4)$$

where b is a given threshold of a maximum number of gates to virtualize to control the maximum overhead from gate virtualization. The second term of the minimization statement ensures that for multiple solutions that minimize the qubit dependencies, we find the one that uses the least amount of gate virtualizations.

3.2 Design Challenges and Key Ideas

Challenge #1: Programmability and generality. The promising technique of GV is a new and rather complex concept. It is not trivial how to virtualize two-qubit gates using single-qubit gates or how to keep track of the created circuit fragments. Therefore, we must develop general abstractions that implement the new virtualization techniques and allow simple, automatic application to quantum circuits while allowing straightforward integration into existing transpilation and optimization infrastructures.

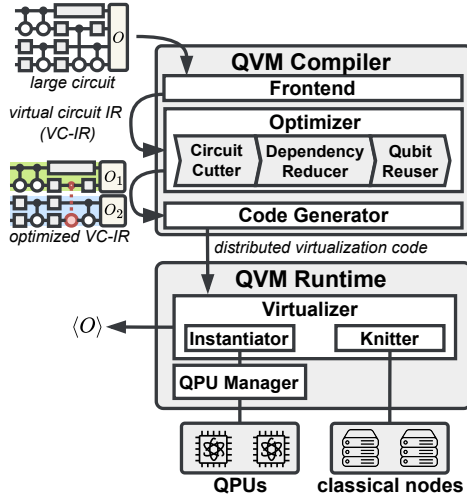


Fig. 3. Overview of the QVM framework (§ 3.3). The Quantum Gate Virtualization Machine (QVM) consists of two main components: QVM Compiler and QVM Runtime.

Approach: Virtual circuit IR (VC-IR) as an intermediate representation: We introduce the *virtual circuit IR* (VC-IR) to enable a unified optimization and execution process of large circuits using gate virtualization. The VC-IR is an intermediate step between any high-level circuit representation and smaller optimized circuit fragments.

Challenge #2: Fidelity. The noisy circuit executions on QPUs hinder the practicality of current quantum algorithms. Every operation applied on a qubit incurs noise to the final result, which propagates and amplifies throughout the circuit. To ensure higher fidelity in quantum computations, it is essential to employ procedures that optimize the circuit's structure using the promising technique of gate virtualization. This involves decomposing the circuit into smaller fragments, reducing the circuit's depth, and minimizing the number of non-local operations or qubit dependencies while minimizing the overhead of virtualization.

Approach: A compiler for optimal gate virtualization: We introduce the *QVM Compiler*, a modular architecture designed to compile circuits utilizing gate virtualization. The compiler converts a quantum circuit into a VC, applies a customizable series of optimization passes on the VC to take advantage of gate virtualization opportunities, and prepares the VC for execution on a set of QPUs.

Challenge #3: Scalability. Gate virtualization incurs an exponential overhead of $O(6^k)$ for k virtual gates, both in quantum computation and in classical postprocessing (§ 2.3). This overhead appears since we need to execute the fragments in $O(6^k)$ instantiations, and then we need to post-process all instantiation results to compute the final result. To maximize the possible gate virtualizations, it is crucial to implement highly parallel computation on multiple QPUs and classical processors.

Approach: A parallel scalable runtime: We present *QVM Runtime*, a scalable system for executing virtual circuits. The runtime efficiently instantiates the high amount of fragments, distributes them between available QPUs for parallel quantum processing, and uses a highly scalable parallel process to post-process the fragment results.

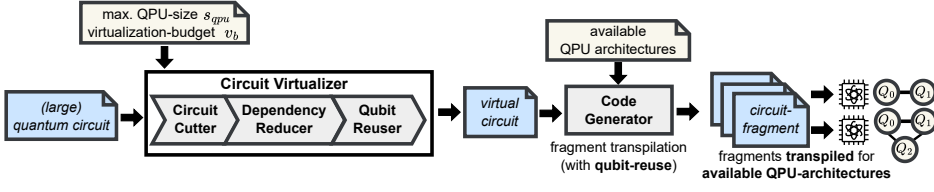


Fig. 4. Overview of the QVM Compiler (§ 4). The QVM Compiler consists of three stages: the Transformer (virtual circuit generation), Optimizer (a modular compiler optimization workflow), and Code Generator (transpilation for target QPUs).

3.3 The QVM Framework

Based on the aforementioned key ideas, we propose the design of our Quantum Gate Virtualization Machine (QVM) framework, an end-to-end system that exploits the full potential of gate virtualization to achieve scalable execution of large circuit with high fidelity (see Fig. 3). The QVM system builds on the abstraction of a *virtual circuit* to utilize gate virtualization. It consists of two main components: the QVM Compiler (§ 4) and the QVM Runtime (§ 5). As QVM operates at the circuit-level abstraction, it is entirely hardware-independent and compatible with any QPU type.

QVM Virtual circuit IR (VC-IR). The virtual circuit (VC) abstraction extends the traditional quantum circuit abstraction (§ 4.2). For this, it incorporates the abstraction of *virtual gates* and views the circuit as a collection of circuit *fragments*, where each fragment is a circuit acting on a subset of qubits of the original circuit.

QVM compiler. The QVM compiler (Fig. 3, top) is responsible for compiling a quantum circuit efficiently to a set of smaller circuit fragments by using gate virtualization. The compiler operates in three stages: (1) the *frontend* converts the circuit into the VC-IR, (2) the *virtual circuit optimizer* applies gate virtualization to reduce circuit depth, width, and/or intra-circuit dependencies, and (3) the *code generator* prepares the circuit fragments for execution on a set of QPUs. For the virtual circuit optimizer, we describe the implementation of three optimization passes of the *circuit cutter*, the *dependency reducer*, and the *qubit reuser*, which are designed to optimize arbitrary virtual circuits. Additionally, users can easily plug in their own optimization passes, which modify a virtual circuit, e.g., to efficiently optimize specific circuits of a known structure.

QVM runtime. The QVM runtime (Fig. 3, bottom) is the system responsible for the scalable execution of virtual circuits. The runtime consists of two components: the *virtualizer* and the *scheduler*. The virtualizer is responsible for implementing the gate virtualizations according to Fig. 1, using fragment instantiation and parallel post-processing. The QPU manager is responsible for the parallel execution of the fragments on a set of QPUs.

4 The QVM Compiler

We now describe the design and implementation of our QVM compiler. The QVM compiler is an extensible pipeline for the efficient virtualization of gates and to prepare a large circuit for executing a set of small QPUs.

4.1 Workflow of the QVM Compiler

Fig. 4 shows the workflow of the QVM compiler. First, the **frontend** of our compiler takes a (large) quantum circuit and converts the circuit into the virtual circuit IR (VC-IR) (Fig. 5).

Then, the VC-IR is optimized using the **optimizer**. Each compiler optimization pass receives two inputs: the maximum fragment size s , which specifies the maximum width each fragment must have, and a virtualization budget b , which constrains the number of allowed gate virtualizations to limit the maximum virtualization overhead. Typically, we choose s as the size of the largest

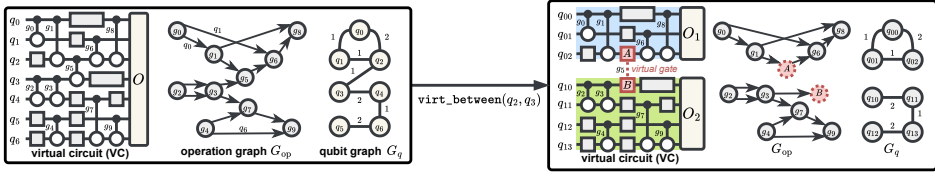


Fig. 5. Virtual Circuit IR (VC-IR) (§ 4.2). A virtual circuit (VC) extends a quantum circuit by incorporating virtual gates and managing the qubits in sets of fragments. The VC-IR manages the operation graph G_{op} and the qubit graph G_q for efficient analysis and manipulation with the VC-IR API to apply GV.

available QPU to ensure every fragment is executable by at least one QPU. For our optimizer, we design a pipeline of the following three generic optimization passes:

#1: Circuit cutter (CC): First, the circuit cutter (CC) pass (§ 4.3) aims to decompose the VC into fragments smaller than s , using $v \leq b$ virtual gates, and reduces the budget to $b = b - v$. If CC fails to decompose the circuit within the given budget, no gate is virtualized.

#2: Dependency reducer (DR): In the case where the budget b is not yet exhausted by the circuit cutter pass, the dependency reducer (DR) (§ 4.4) applies at most the remaining virtualization budget of b gate virtualizations to reduce the dependencies between qubits and operations within the VC to reduce noise propagation and depth.

#3: Qubit reuser (QR): Lastly, the qubit reuser (§ 4.5) reuses qubits within individual fragments to further reduce circuit width if the CC fails to reduce the fragment sizes sufficiently. If the qubit reuser fails to reduce the width of any fragment to s , the optimization pipeline fails.

After the optimization phase, the **code generator (CG)** acts as the backend of our compiler (§ 4.6) by extracting the fragments as parameterized circuits, optimizing the circuits and generating the inputs for the instantiation of each fragment.

Next, we describe the QVM compiler stages in detail.

4.2 Virtual Circuit IR and Frontend

To enable easy integration and a simple workflow for gate virtualization during compilation and runtime, QVM provides the virtual circuit IR (VC-IR) (Fig. 5). In total, the VC-IR provides three main data structures: (1) A *virtual circuit* (VC), which can contain *virtual gates* and consists of several *fragments*, (2) an operation graph G_{op} and (3) a qubit graph G_q , as described below:

Virtual circuit and virtual gates. A VC extends the traditional abstraction of a quantum circuit by additionally incorporating the functionality of consisting of a set of fragments and allowing *virtual gates* to be part of its instructions.

A *fragment* describes subcircuit consisting of a subset of the operations of the original circuit that operate on a subset of qubits that are not connected to other qubits in the VC via a real two-qubit gate. We implement fragments by using a separate qubit register for each fragment.

A *virtual gate* expresses the notion of the virtualization of a two-qubit quantum gate (§ 2.3). A virtual gate is a two-qubit gate that does not require a real connection between its two qubits. Therefore, a conventional transpiler or circuit optimizer would treat a virtual gate as two one-qubit gates. Hence, a virtual gate has no influence on, e.g., the assignment and routing of qubits. A virtual gate can be split into two one-qubit gates, whose instantiations are inserted during execution (§ 5.2).

Operation graph. The operation graph G_{op} expresses the gate dependencies of the VC as a directed acyclic graph (DAG). G_{op} is a graph in which the vertices are the two-qubit gates of the circuits, and the edges represent the direct dependencies between the respective operations via a qubit wire. Therefore, each edge contains the respective qubit as an attribute.

Qubit graph. To efficiently represent the connections between qubits of a VC, we utilize the representation of a qubit graph G_q , where the qubits are the vertices. An edge exists between two

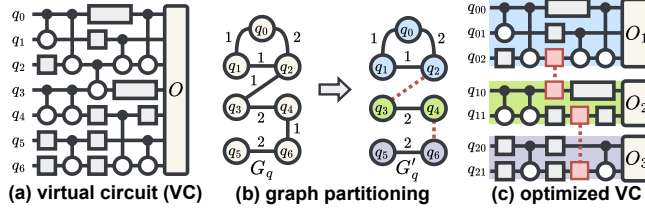


Fig. 6. Circuit Cutter (CC) (§ 4.3). The CC receives a large virtual circuit (VC) with $n_q = 6$ qubits and performs a graph partitioning on the qubit graph G_q to dissect the VC into fragments of size $s = 3$ by inserting virtual gates between the partitions.

qubits when the qubits are connected with at least one two-qubit gate. So, the connected subgraphs of G_q directly correspond to the VC's fragments. Each edge holds a weight with the number of two-qubit gates between the two qubits.

Gate virtualization API. To efficiently virtualize gates, the VC-IR exposes two main functions:

- `virt_gate( $g_x$ )`: Virtualizes the gate g_x , removes g_x from G_{op} and adds single-qubit gates instead. Decrements the weight on the edge (q_i, q_j) in G_q , where g_x acts on q_i and q_j .
- `virt_between( $q_i, q_j$ )`: Virtualizes every gate which acts on the qubits q_i and q_j . Removes the edge (q_i, q_j) from G_q , and updates G_{op} accordingly.

Fig. 5 shows an example of calling `virt_between( $q_i, q_j$ )`.

Compositionality. The VC-IR abstraction allows for full composition. By simply adding more fragments to the circuit, two VCs can be combined, either connected via an explicit virtual gate or completely independent from one another. The QVM compiler and runtime would handle this composed circuit as a single, larger VC.

Frontend: Virtual circuit generation. The frontend of the QVM compiler generates the VC-IR from an input circuit. The VC is initially a copy of the original circuit, i.e. a VC that consists of one fragment and no virtual gates. We generate G_{op} and G_q by traversing the operations of the circuit.

To enable efficient transpilation and execution of virtual circuits, the virtual circuit allows fragments to be easily extracted and replaced (Figure 5 (b)). To extract a fragment as a circuit, the virtual gates acting on the qubits of the respective fragment are decomposed into one-qubit placeholder gates, and the circuit consisting of the gates acting on the qubits of the fragment is returned. The gates of the fragment can then be modified (e.g., optimized) and the respective fragment can be replaced in the virtual circuit. In this way, it is possible, for example, to transpile the individual fragments for certain QPUs only once (§ 4.6) and insert the various instantiations of the virtual gates only shortly before execution, without the need for additional and repeated transpilation of the fragment circuits (§ 5.3).

4.3 Circuit Cutter (CC)

The aim of the Circuit Cutter (CC) optimization pass is to split the VC into several fragments so that each fragment has s or fewer qubits, while using as minimal virtual gates as possible to minimize the computational overhead (Fig. 6). For this purpose, the CC performs a graph partitioning on the qubit-graph G_q as follows:

Circuit cutter model. We assign the vertices $q_x \in V_q$ of the qubit graph $G_q = (V_q, E_q)$ into at least $f = \lceil n_q/s \rceil$ subsets F_j . According to this mapping, $E_{cut} = \{(q_x, q_y) : q_x \in F_j, q_y \in F_i, F_j \neq F_i\}$ is the set of all edges that need to be removed to decompose the G_q into independent subgraphs. In our cutting model, we find a solution that minimizes $\sum_{(q_x, q_y) \in E_{cut}} w(q_x, q_y)$, where $w(q_x, q_y)$ is the weight of the respective edge $(q_x, q_y) \in E_{cut}$. Amongst all possible optimal solutions that amount to the weight, we choose a solution that minimizes $\sum_j |F_j|^2$, such that we favor the solution

that distributes the number of qubits evenly across the fragments. The subsets F_j correspond to fragments of the resulting optimized VC.

Algorithm. In Figure 6 we show an overview of the circuit cutter (CC). The CC inserts virtual gates such that the fragments of the resulting virtual circuit can be executed on smaller QPUs with at most s qubits. In doing so, it aims to minimize the number of virtual gates to reduce the induced overhead. To this end, CC takes the following steps:

Step 1: We convert the circuit into a representation of a qubit graph G_q (Figure 6, middle). In G_q , each node represents a qubit in the circuit and each edge represents a connection between two qubits via two-qubit gates. Each edge has a weight indicating the number of two-qubit gates between the two qubits.

Step 2: We partition G_q into at least $f = \lceil n_q/s \rceil$ disjoint qubit sets P_x , where n_q is the number of qubits in the circuit and each qubit set has at most s qubits and the sum of the edge weights between the sets is as few as possible. For this purpose, we can use any graph partitioning algorithm.

Step 3: For each qubit pair where the qubits are in different partitions $P_x \neq P_y$, we virtualize every two-qubit gate acting on that qubit pair. In this way, we can decompose the circuit into a set of fragments according to the graph partitions of G_q . If we cannot find a solution for partitioning the circuit into fragments smaller than s and requiring only b virtual gates or less, CC does not modify the original virtual circuit.

Within QVM, we implement two graph partitioning procedures for the CC: First, a **greedy graph partitioning** using a recursive Kernighan-Lin bisection [37], is used for large circuits with an arbitrarily large number of qubits. This procedure recursively bisects the current largest partition until each partition has a size less than or equal to s_{qpu} .

We implement the model with Answer Set Programming (ASP) using the Clingo solver to find an optimal solution [24, 68]. For each $(q_x, q_y) \in E_{cut}$ we call `virt_between( $q_x, q_y$ )` to update the VC-IR according to the solution of the model.

Greedy Circuit Cutter. In addition to the procedure that finds an optimal solution for our model, we also implement a CC that uses an efficient heuristic approach based on the greedy Kernighan-Lin bisection algorithm [37] to enable shorter compilation times for large circuits. To decompose a VC into multiple fragments of size s or less, we iteratively apply the Kernighan-Lin bisection to the currently largest connected subgraph of G_q . The bisection determines two distinct sets of vertices V_1 and V_2 such that $|V_1| \approx |V_2|$, and the sum of weights of the edges between the two sets of vertices is as minimal as possible. Then we call `virt_between( $q_x, q_y$ )` for each (q_x, q_y) , where $q_x \in V_1$ and $q_y \in V_2$. We apply this iteration until each fragment of the VC has less than s qubits.

Note that in the partitioning algorithms for gate virtualization, the search space scales only one-dimensionally with the number of qubits in the circuit and not also with the number of two-qubit gates in the circuit, as is the case with wire-cutting [83]. Since the number of two-qubit gates in a circuit is typically much larger than the number of qubits, our gate-cutting techniques are generally much more efficient than their wire-cutting counterparts. This makes optimal graph partitioning for gate virtualization a suitable option for current quantum algorithms with hundreds of qubits and few partitions, where the circuit cutting time is negligible compared to execution time.

4.4 Dependency Reducer (DR)

The Dependency Reducer (DR) reduces the number of circuit intra-dependencies by using as few virtual gates as possible. The dependencies between qubits and operations are best illustrated with the VC-IR's operation graph G_{op} . An example of a G_{op} in the context of qubit dependencies is shown in Fig. 7 (a). In this example, every qubit q_i is dependent on every other qubit q_j , since some gate g_x acting on q_i depends on a gate g_y acting on q_j [31]. This means that noise occurring on one qubit could also propagate to all other qubits in the circuit, amplifying overall errors.

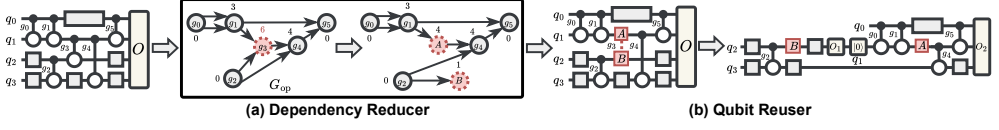


Fig. 7. Dependency Reducer (DR) and Qubit Reuser (QR) (§ 4.4-4.5). (a) The greedy DR iteratively virtualizes gates to reduce the number of qubit-dependencies in a circuit. (b) Because of reduced qubit dependencies, the QR can reuse qubits to reduce the circuit width.

As shown in Fig. 7, the DR can reduce the intra-dependency of the circuit by inserting virtual gates into the circuit while being constrained by the budget b of the maximum gate virtualizations. To reduce the qubit-dependencies as efficiently as possible, we adhere to the following model:

Dependency reducer model. We aim to minimize the number of qubit-dependencies by virtualizing gates in the VC-IR. A qubit q_i is dependent on another qubit q_j if there exists a path in G_{op} from a gate g_x acting on q_j to a gate g_y acting on q_i . Let $D_q = \{(q_i, q_j) : q_i \text{ depends on } q_j\}$ be the set of all qubit-dependencies. We need to find a set G_{virt} of gates that, when removed from G_{op} , minimize the number of qubit dependencies $|D_q|$ the most. If multiple optimal solutions exist, we choose a solution that minimizes $|G_{virt}|$. We implement this model using Answer Set Programming (ASP) and use the Clingo solver to solve for an optimal solution [24, 68].

Greedy dependency reducer. For circuits with a large number of gates, we additionally design an efficient greedy DR (G-DR) algorithm (Fig. 7). The algorithm works as follows:

First, we determine the most critical two-qubit gate in the circuit, i.e., the two-qubit gate that, when virtualized, can reduce the most intra-circuit dependencies. For this, we label every two-qubit gate g_x in the circuit with cost d_i . This cost is defined as $d_i = anc(g_x) \cdot desc(g_x)$, where $anc(g_x)$ is the number of ancestors, and $desc(g_x)$ is the number of descendants of g_x . Therefore, the gate with the highest cost depends on most other gates in the circuit and is, therefore, most likely responsible for a large amount of qubit and gate dependencies. Then, we call $virt_gate(g_x)$ on the most-critical gate g_x to virtualize the gate in the VC and remove the two-qubit gate from G_{op} . If two or more gates have the same cost, we choose a gate randomly. We decrement the budget b , and repeat the process until $b = 0$.

In the example from Fig. 7 (a), the G-DR would virtualize g_3 first, since it has the highest single cost of $d_3 = anc(g_3) \cdot desc(g_3) = 3 \cdot 2 = 6$. In this single iteration of G-DR, we can reduce the number of qubit dependencies from 12 to 11 since now q_2 does not depend on q_1 anymore. This means that errors of q_1 cannot propagate to q_2 . It also reduces the cost of all the gates in the circuit, meaning that the gates depend on significantly fewer other gates and are less likely to amplify overall noise.

Note that our G-DR computes the number of ancestors for each node in a single traversal of G_g in topological order. Similarly, the number of descendants of each node is computed in a single traversal in reversed order. Therefore, the time complexity of G-DR is $O(2 \cdot n_v \cdot |V_g|)$, where $|V_g|$ is the set of nodes in G_g . Thus, the algorithm has linear time complexity in the number of gates.

4.5 Qubit Reuser (QR)

In the final pass of the optimizer, we apply the qubit reuser on individual fragments to reduce their width further, in case their width still exceeds the maximal size s . To this end, the qubit reuser first checks whether each fragment in the VC has a width of s or less. For each fragment with a width greater than s , the qubit reuser applies a qubit reuse procedure to reduce the width to s to ensure that each fragment can execute on the available QPUs. We can reuse a qubit q_i for another qubit q_j if q_i does not depend on q_j by inserting a mid-circuit measurement and resetting the qubit [31, 35]. Since the computation on qubit q_i does not depend on qubit q_j , we can first complete all operations involving q_j . After completion, we reset q_j and reuse it to perform the computation

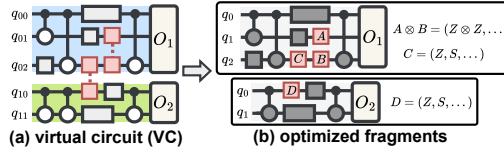


Fig. 8. Code Generator (CG) (§ 4.6). The CG generates parallel virtualization code as parameterized fragments.

originally intended for q_i . Fig. 7 (b) shows this qubit reuse pass, where we can reuse q_2 for q_1 since q_2 does not depend on q_1 .

The QR builds upon the foundations of qubit reuse [31] and recycling [35], as discussed extensively in previous works. However, note that an appropriate level of qubit reuse may only be possible through the preceding DR pass, which reduces the number of dependent qubit pairs as shown by the example of Fig. 7. This ultimately enables us to combine both techniques, achieving better results in circuit size reduction with lower overhead. Concretely, a similar reduction in width by circuit cutting would have required two virtual gates (or two wire cuts [65, 83]). Therefore, in our example, reducing dependencies and reusing qubits is the most efficient solution in terms of virtualization overhead: we reduce the width to $s = 3$ with a virtualization budget of only $b = 1$.

4.6 Code Generator (CG)

The final step of the QVM compiler is generating the code in the form of circuits, which can be executed by the QVM runtime (Fig. 8). To do so, we first extract each fragment as an individual circuit from the VC by collecting all operations on the respective qubit register. In these extracted circuits, we insert placeholder gates at the qubits of the virtual gates. The placeholder gates are parameterized gates, which can be instantiated with the actual gates that we need to insert to reconstruct the result (§ 2.3). For the instantiation, the CG creates a parameter vector for each placeholder gate, which describes the gates to be inserted for the respective instance of the virtual gate. Finally, the code-generator runs a set of standard circuit optimization passes to optimize the individual circuits. This means the heavy optimization must be executed only once, reducing the just-in-time transpilation time before instance execution.

5 The QVM Runtime

5.1 Workflow of the QVM Runtime

Fig. 9 shows the workflow of the QVM runtime. As the first step, we pass optimized fragment circuits generated by the QVM compiler to the *virtualizer*. Here, the *instantiator* generates the instances of each circuit and passes them together with the fragments to the QPU manager. The *QPU manager* then executes the fragments with each given instantiation on the set of QPUs. The results are returned to the *knitter* component of the virtualizer, where the final result is reconstructed through parallel classical post-processing.

5.2 Virtualizer

The virtualizer implements the logic for executing virtual gates. For this purpose, the virtualizer consists of two components, the *instantiator* (Fig. 9, Step 1) and the *knitter* (Fig. 9, Step 3).

Instantiator. The instantiator is responsible for creating instances of gates that must be inserted into the fragment. For this purpose, the instantiator creates 6^{k_j} instances for each fragment F_j , where k_j is the number of virtual gates that act on the qubits of F_j . This is because we need to execute all combinations of gates that need to be executed to do the gate-decomposition of the gate virtualization (cf. § 2.3). The instances are described as assignments to the parameterized gates and include every possible combination of the total 6^{k_j} combinations of each fragment. These

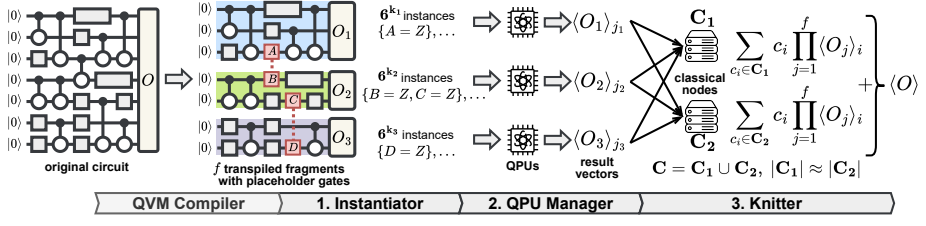


Fig. 9. Workflow of the QVM Runtime (§ 5). (1) The instantiator generates the instantiations inserted into the placeholder gates of the compiled fragments. (2) The QPU manager runs the instances on multiple QPUs in parallel. (3) The knitter reconstructs the probability distribution of the original circuit by merging and then knitting the instances in highly parallelizable steps.

assignments are essentially the tensor-product of the parameter vectors of the generated code for each fragment (Fig. 8).

Knitter. The knitter takes the approximate probability distributions based on the sampling results of all fragment instances and calculates the final result of the original circuit by applying the formulas for gate virtualization with highly parallel processing. (§ 2.3). The results are given as vectors for each fragment F_j with entries $\langle O_j \rangle_i$ with $i = 1, \dots, 6^{k_j}$. To knit the results, the knitter distributes the result vectors of each fragment to the available classical nodes, where each node is given the task of computing a part of the global 6^k instances. We determine this part by assigning an equal part of the global coefficient vector C to each node (Eq. 2.3). In the example of Fig. 9, we divide the coefficient vector C into two parts C_1 and C_2 and calculate the partial sum at each node over the instances corresponding to each coefficient. Finally, we calculate the sum of the two partial results to obtain the final result $\langle O \rangle$. In this way, we are able to linearly scale the post-processing of the circuit virtualization with respect to the number of cores used.

Extensibility. We implement a virtualizer for gate virtualization as presented in [47]. However, the design of our virtualizer also allows us to implement other divide-and-conquer techniques effectively [6, 83]. Such techniques all follow the same workflow of our virtualizer and could, therefore, be easily integrated into the QVM runtime.

5.3 QPU Manager

The QPU manager is responsible for a scalable execution of the 6^{k_j} instances of each circuit fragment F_j on a set of individual QPUs, returning the result-vector for each fragment (Fig. 5, Step 2). For this, the QPU manager receives an optimized circuit fragment (§ 4.6) and all instance combinations generated by the instantiator.

Scheduling algorithm. To execute a fragment, the QPU-manager does the following steps:

Step 1: For each QPU QPU_i with enough qubits to run the circuit, we transpile the circuit to, including mapping and routing on the physical qubits of QPU_i . Note that this has to be done only once for each. We then compute the estimated probability of success $esp(QPU_i)$ of executing the circuit on that QPU. This is done by computing the cost of the errors induced by the gates and measurements on the assigned physical qubits, as described in [53].

Step 2: We normalize the current job queue sizes of the QPUs by dividing the length of each job queue by the length of the maximum job queue. This yields a relative waiting time as $w(QPU_i) \in [0, 1]$, where a higher value means a longer waiting time for the job.

Step 3: We compute the score s_i for each QPU, where $c_i = \alpha \cdot (1 - w(QPU_i)) + \beta \cdot esp(QPU_i)$, and choose the QPU with the highest score to execute the corresponding fragment. The user can choose α and β to provide either fast runtime or less noisy results.

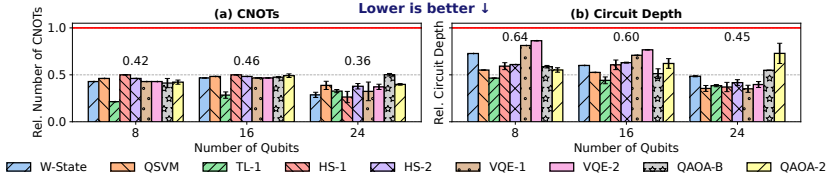


Fig. 10. Circuit Cutter (§ 6.1). Impact of QVM's optimal circuit cutter on number of CNOTs and circuit depth.

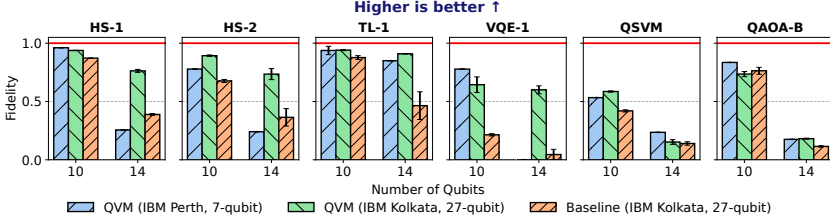


Fig. 11. Circuit Cutter (§ 6.1). Fidelity of running QVM with the circuit cutter on IBM Perth and IBM Kolkata.

Step 4: Finally, for each instance combination, we insert the instantiation into the transpiled fragment for the selected QPU, resulting in a total of 6^{k_j} circuits when k_j gates act in the respective fragment F_j . These circuits are then sent to the QPU as a job for execution.

Our strategy of incorporating queue times and estimated probabilities of success into the QPU manager can be easily applied to the current cloud-centric quantum infrastructure, where our QPU manager would be a client for some quantum resources offered by cloud providers [73]. Our solution is currently the most efficient, as there is little control over the cloud's internal queues.

6 Evaluation

Experimental setup. We conduct three types of experiments: (1) circuit transpilation with and without QVM's compiler to measure the circuit's properties post-compilation, (2) runs on real QPUs for measuring the circuit's fidelity, and (3) classical simulation of large circuits cut into fragments of different sizes. For (2) we conduct our experiments on Falcon r5.11H QPUs, namely the 7-qubit IBM Perth and the 27-qubit IBMQ Kolkata. For (1) and (3) we use the Qiskit Transpiler and Qiskit Aer, respectively, and run on our local classical machines. For classical post-processing and simulation we use a server with a 64-core AMD EPYC 7713P processor and 512 GB ECC memory.

Framework and configuration. We use the Qiskit [70] Python SDK version 0.41.0 for quantum circuits and simulations. We transpile any quantum circuit we run with the highest optimization level O3 and run with 20,000 shots. To get a meaningful measurement of the fidelity or circuit properties on real QPUs, we run QVM only on a single QPU. When we benchmark the performance of the QVM runtime with simulators, we utilize every system core.

Benchmarks. We study QVM on a set of circuits used in the state-of-the-art benchmark suits Supermarq [88], MQT-Bench [71], and QASM-Bench [40]. These circuits can be scaled both in the number of qubits and depth. Specifically, we study: W-State, Bernstein Vazirani (BV), Quantum Support Vector Machine (QSVM), Hamiltonian Simulation (HS- t), Two Local Ansatz (TL- n) with circular entanglement, Variational Quantum Eigensolver (VQE- n) with a Real-Amplitudes ansatz of linear entanglement, Approximate Optimization Algorithm (QAOA- d) with regular graphs of degree d and barbell graphs (QAOA-B). HS, VQE, and TL are scalable in their circuit layers t or n .

Metrics. We evaluate the following metrics.

- **Fidelity:** We use the Hellinger fidelity $\left(1 - H(P_{ideal}, P_{noisy})^2\right)^2 \in [0, 1]$ to measure how close a noisy result is to the desired ground truth of a quantum circuit. Here, H is the Hellinger distance between two probability distributions [30].



Fig. 12. Circuit Cutter vs. CutQC (§ 6.1). Relative number of CNOT gates and fragment depth after compiling with QVM vs. cutting with CutQC on IBM Kolkata.

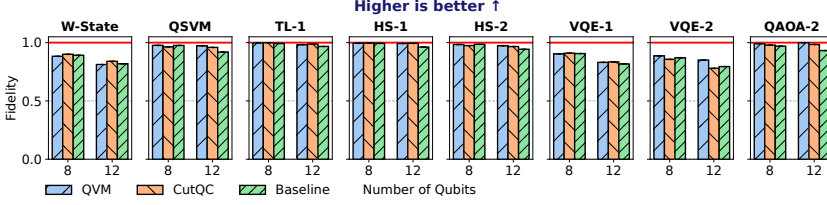


Fig. 13. Circuit Cutter vs. CutQC (§ 6.1). Fidelities of running QVM vs. CutQC vs. Qiskit on IBM Kolkata.

- **Circuit properties:** Number of CNOT gates, depth and the number of qubit dependencies. When a VC contains more than one fragment, we use the fragment with the *worst* property (i.e., maximal depth, dependencies, number of CNOTs)
- **Execution time:** The execution time of a VC in seconds.
- **Estimated success probability:** We use the estimated success probability (ESP) metric to measure the estimated fidelity on larger quantum systems. We define the ESP as $\prod_i (1 - e_i)$, where e_i is the error of the i -th operation in the circuit [52]. If a VC has multiple fragments, we report the minimum ESP.

Note that we report relative values as the ratio $v_{\text{QVM}}/v_{\text{baseline}}$, where v_{QVM} is the absolute value from QVM and v_{baseline} is the corresponding baseline value.

Baseline. We use the Qiskit transpiler with O3 [1] and CutQC [83] as our baselines for circuit compilation and runtime evaluation. CutQC [83], building on the framework proposed by Peng et al. [65], deals with another widely studied way of cutting circuits, namely wire cutting, which, unlike cutting two-qubit gates, allows decomposing the time evolution of individual qubits. This is achieved by cutting qubit wires to divide the circuit into several sub-circuits. To reconstruct the result of the original circuit, the sliced qubit is measured and reinitialized in four different computational bases, followed by classical post-processing with an exponential overhead [47, 65].

6.1 Circuit Cutter

RQ1: How well does QVM’s circuit cutter allow scaling of circuits that can run on noisy QPUs with acceptable fidelity? We evaluate the impact of the circuit cutter on the CNOT count and depth of transpiled circuits and the fidelity of running virtual circuits using our optimal graph partitioner.

Impact on number of CNOTs and circuit depth. In Fig. 10, we study the maximum number of CNOTs and circuit depths of the fragments after compilation with our circuit cutter with a maximum of three virtual gates. Each virtual circuit is decomposed into fragments of a maximum of 13 qubits, and the fragments are transpiled for the 27-qubit IBMQ Kolkata QPU. The results in Fig. 10 (a) show that the number of CNOTs decreases by 41% on average. Fig. 10 (b) shows that the circuit depth decreases by 56% on average. This shows that it is possible to almost double the size of the high-fidelity circuits since the number of CNOTs and circuit depth is approximately halved.

Impact on fidelity. The impact of using QVM on the fidelity of the execution is shown in Fig. 11. Here, the circuit cutter decomposes the circuits into fragments of maximally 7 qubits to fit the

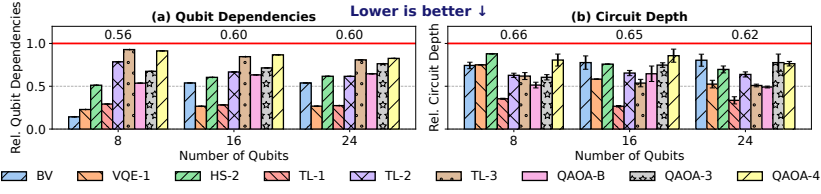


Fig. 14. Dependency Reducer (§ 6.2). Impact of the greedy qubit dependency reducer on (a) the number of qubit dependencies and (b) on the circuit depth of the transpiled circuit for IBM Kolkata. We use at most three virtual gates to compile the circuit.

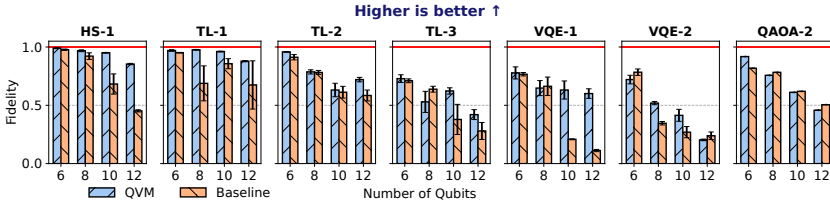


Fig. 15. Dependency Reducer (§ 6.2). Fidelity of the greedy dependency reducer on IBM Kolkata.

small 7-qubit IBM QPUs. The fragments are run on both the 7-qubit IBM Perth and the 27-qubit IBM Kolkata QPUs, and compared to the baseline fidelity of running the circuits on IBM Kolkata. We run the experiment for various benchmarks with sizes of 10 and 14 qubits. We observe that the fidelity of running the circuit on IBM Kolkata improves the fidelity by 4.7× on average and up to 33.6×. E.g. for the VQE-2 benchmark, the fidelity of the benchmark diminishes, while QVM can still create higher fidelities. Compared to the baseline, running QVM on the IBM Perth improves the fidelity by 2.1× on average and up to 10.6×. Therefore, we show that QVM can reliably simulate a larger QPU using smaller noisy QPUs while producing higher fidelity. This is despite IBM Perth having a median of 2.3× higher readout and 1.2× higher CNOT error during our experiments.

Comparison to CutQC. In Fig. 12 and 13 we compare the circuit cutter of QVM with CutQC [83]. We run the QVM and CutQC circuit cutters with the same configuration to generate circuit fragments of up to 7 qubits and compile and run the fragments on the IBM Kolkata QPU. We use several benchmarks of sizes of 8-12 qubits. We find that, compared to CutQC, QVM only produces 70% of the CNOTs on average, since gate virtualization allows a reduction of the qubit connectivity significantly compared to CutQC (Fig. 12 (a)). QVM achieves similar circuit depth reduction as CutQC as both can cut the circuits into significantly smaller fragments (Fig. 12 (b)).

A look at the fidelity benchmark (Fig. 13) shows that CutQC and QVM achieve similar fidelity and significantly outperform the Qiskit baseline, with QVM achieving on average 1% higher fidelity than CutQC. We suspect the relatively small improvement despite the promising results in circuit properties is due to the noisy mid-circuit measurements with an error of $\geq 10^{-2}$, which we need to perform to virtualize gates. With less measurement noise, QVM will perform similarly to CutQC.

We conclude that both QVM and CutQC, with their different techniques, are efficient in-circuit cutting and should ideally be used together in future work to take advantage of both methods with their respective benefits [11], especially when we mitigate the mid-circuit measurement errors [29].

RQ1 takeaway: With the circuit cutter, we reliably scale the size of circuits that can be run on noisy QPUs, up to 2×, improving the overall fidelity 4.7× on average and up to 33.6× due to significant depth and CNOT gate reduction.

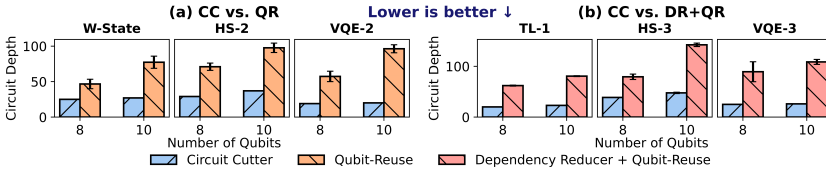


Fig. 16. Qubit Reuser (§ 6.3). Depths of compiled circuits with a maximal fragment size of 5 transpiled for IBM Perth. (a) Circuit cutter vs. qubit reuser. (b) Circuit cutter vs. dependency reducer and qubit reuser.

6.2 Dependency Reducer

RQ2: By how much does the dependency reducer (DR) decrease the number of dependencies within the circuit, improving the fidelity of running the circuit on noisy QPUs? For this experiment, we evaluate DR with a maximum of three virtual gates on differently sized benchmarks on IBM Kolkata.

Impact on qubit dependencies and circuit depth. Fig. 14 (a) shows the effect of DR on the number of qubit dependencies in the logical circuit, compared to the baseline of the circuit without DR. On average, the number of qubit dependencies decreases by 58%. This shows that the DR can effectively resolve the dependencies between qubits, reducing noise propagation through the circuit. As Fig. 14 (b) shows, the depth of the circuits transpiled for IBM Kolkata decreases significantly by 64% on average. This is due to the transpiler having fewer constraints on circuit mapping and routing after applying DR, resulting in a transpiled circuit with less depth.

Impact on fidelity. We analyze the fidelity of our baseline and compared it to the DR in Fig. 15, utilizing only one virtual gate. Our results indicate an average increase in fidelity of 36% and up to 5.2×. However, the noisy mid-circuit measurements needed for gate virtualization could limit the improvement in fidelity. These measurements typically induce significant noise, which affects the overall fidelity of virtual circuit execution [79, 95].

RQ2 takeaway: The DR decreases the dependencies between qubits by 58% and circuit depth by 64% using at most three virtual gates. This also leads to an average increase in fidelity by 36% and up to 5.2×, using only one virtual gate.

6.3 Tradeoffs with the Qubit Reuser

RQ3: What is the effect of using the qubit reuser (QR) to reduce the width of the circuit fragments further? We show the trade-offs of using the CC alone against the DR and QR to reduce the width of circuits to run on small QPUs. To show this tradeoff, we compile circuits with different optimizer configurations, such that each fragment's width is maximally five qubits. We select benchmarks ensuring each technique can reduce the qubit count to the target of five.

Circuit-cutter vs. Qubit-reuse. In Fig. 16 (top), we compare the effects of using either the CC or the QR to reduce the width of a virtual circuit on the circuit depth of the transpiled fragments. Our results show that the CC compiles the circuits to only 37% compared to QR on average. This is because the CC can break down the circuit into smaller fragments with reduced width while only incurring a maximum of two virtual gates. The QR, however, increases the depth of the circuit substantially while reusing qubits, which in turn will reduce overall fidelity. So, there is a tradeoff between using gate virtualization to reduce the depth against using qubit reuse without overhead.

Combining dependency reducer and qubit-reuse. In Fig. 16 (bottom), we show how the CC pass compares to the DR and QR passes to reduce circuit width. For this, we choose benchmarks where, without our DR, qubit-reuse would be impossible since every qubit depends on every other qubit in the circuit. We apply the QR on the reduced-dependency circuit produced by the DR. Like before, we aim to reduce the circuit width to five qubits. The CC uses at most three, and the DR uses one virtual gate, with a 36× lower virtualization overhead. Although using DR & QR incurs

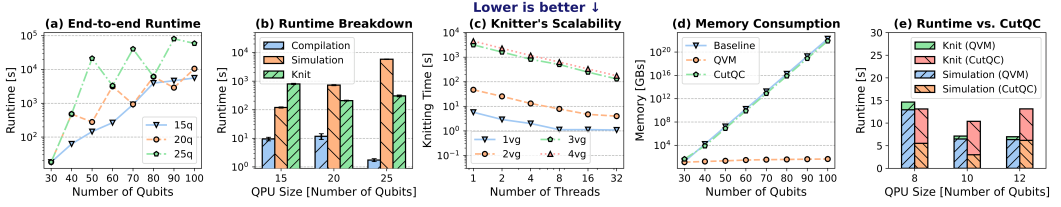


Fig. 17. QVM end-to-end runtime analysis (§ 6.4). (a) End-to-end runtime of 30-100 qubits with different QPU sizes. (b) Runtime breakdown for 70 qubits with different QPU sizes. (c) Knit-time dependent on the number of parallel threads for different numbers of virtual gates (vg). (d) Memory consumption (estimates) for 30-100 qubits compared to the Baseline (statevector simulator) and Full Definition Query CutQC [83] with 20 qubits QPU size. (e) Runtime Comparison against CutQC for 20-qubit circuits.

a low overhead, it also leads to a significantly higher depth than CC. This means that the virtual circuit using DR & QR has 3.2× more depth, negatively impacting fidelity.

RQ3 takeaway: We find a trade-off between overhead and noise when using the CC or DR & QR to reduce the width of quantum circuits. While the CC produces circuits with smaller depths, combining DR and QR allows lower virtualization overhead.

6.4 QVM End-to-end Runtime Analysis

RQ4: How scalable is QVM's runtime and how does QVM compare to classical simulations without cutting & knitting and CutQC? We study the HS-1 benchmark and use the circuit cutter (CC) to compile a VC for a QPU of up to s qubits.

Fig. 17 (a) illustrates the end-to-end runtime needed to simulate *HS-1* after cutting the circuit into fragments that fit QPUs of sizes $s \in \{15, 20, 25\}$. As the full circuit size increases, the runtime also increases, but the growth rate varies among fragment sizes. The smallest fragment size is the fastest, as the simulation overhead outweighs the knitting overhead, even if the circuit has 100 qubits and is cut with five virtual gates. This is evident in Fig. 17 (b) as well, which shows the runtime breakdown for simulating the 70-qubit *HS-1*. As s increases, there is a shift in the runtime from knitting to simulation time. The compilation time remains relatively constant.

Fig. 17 (c) shows the scalability of the knitter (§ 5.2) with its parallelism. We generate knit workloads for 1-4 virtual gates for the 70-qubit *HS-1* benchmark and scale the number from 1 to 32 threads. We observe near-linear scalability with an increasing number of threads, allowing a speedup of up to 25.6× for 32 threads.

We show the memory required to simulate *HS-1* with a chosen QPU size of 20 qubits in Fig. 17 (d). While the baselines, Qiskit Aer statevector, and CutQC with full definition query [83], exhibit exponentially growing memory for linearly increasing circuit sizes, QVM maintains a slightly increasing memory requirement by utilizing sparse quasi-probability distributions. In contrast, CutQC and simulations operate on tensors that need to cover the entire sample space.

Finally, in Fig. 17 (e), we compare the runtimes of QVM and CutQC. We are limited to comparing on small examples, due to CutQC's memory limitations. In particular, we perform 20-qubit circuits for *HS-1* with simulated QPUs of 8-12 qubits. We observe similar runtimes for the QPU size of 8 qubits, as QVM spends more time to simulate a larger number of circuits due to the higher circuit cost [65, 83]. However, with a QPU size of 10 and 12 qubits, QVM clearly outperforms CutQC, as it achieves a significant acceleration in knitting due to its more efficient memory utilization.

RQ4 takeaway: QVM enables simulating large circuits on classical simulators. It can handle circuit sizes of up to 100 qubits or five virtualized gates while maintaining acceptable runtime (~1.5 hours) and very low memory consumption. QVM's knitter allows it to scale linearly.

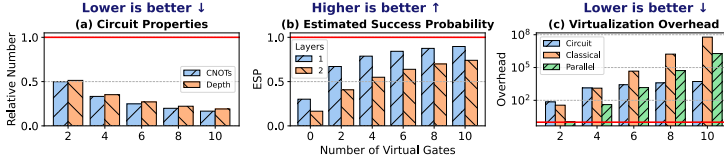


Fig. 18. QVM at practical scale with 500 qubit VQE circuits (§ 6.5). (a) Relative number of CNOTs and circuit depth of the compiled VQE-2 benchmark. (b) Estimated success probability (ESP) with VQE-1 and VQE-2. (c) The overheads of circuit instances and classical postprocessing with and without parallel processing on 32 cores.

6.5 QVM at Practical Scale

RQ5: How does QVM behave on a practical scale with circuits of hundreds of qubits? We would need hundreds to thousands of high-fidelity qubits to demonstrate quantum advantage. However, current QPUs of any hardware type that have up to hundreds or thousands of qubits cannot reliably execute circuits with tens of qubits and higher depth [4, 33, 34, 38]. To investigate how QVM would behave on a practical scale, we evaluate the impact of QVM on 500-qubit VQE circuits on a heavy-hex lattice QPU with 883 physical qubits, which is the typical chip layout for current IBM QPUs [33].

Impact on number of CNOTs and circuit depth. In Fig. 18 (a) we show the effects of the number of CNOTs and the circuit depth of the VQE-2 benchmark. We see that using a budget of two virtual gates reduces the number of CNOTs and the circuit depth by 2×, and using up to 10 virtual gates reduces the numbers by 6×. We see a diminishing improving impact on higher budgets.

Impact on estimated success probability. Fig. 18 (b) shows the estimated success probability (ESP) of the benchmarks VQE-1 and VQE-2. We find that the baseline without virtual gates (shown with 0 virtual gates) achieves an ESP of only 30% and 16%, respectively, which leads to unusable results. When using only two virtual gates, the ESP more than doubles and shows improvements, reaching 90% and 74% with 10 virtual gates. This shows that with QVM, we only need a handful of virtual gates to significantly improve the ESP, leading to usable results.

Impact on processing overheads. The virtualization costs incurred using virtual gates to improve circuit fidelity are shown in Fig. 18 (c). The number of circuits that need to be instantiated and executed increases exponentially with a small number of virtual gates but then only starts to grow linearly with the number of fragments since we only instantiate as many circuits as correspond to the number of gates in the respective fragment (§ 5.2). The classical post-processing overhead grows exponentially with $O(6^k)$, meaning that adding two more virtual gates in the same configuration results in a runtime increase of 36×. Since the QVM runtime provides an almost linear speedup (§ 6.4), we can distribute the knitting across dozens of cores, which significantly mitigates this overhead for a small number of 4-6 virtual gates. This is shown in Fig. 18 (c) as an example of (perfect) linear scaling in classical post-processing with 32 cores.

RQ5 takeaway: For large-scale algorithms, QVM achieves high ESP while using only a handful of virtual gates for which our runtime can achieve significant speedups through parallelization. We therefore find a trade-off between fidelity and quantum-classical co-processing resources.

7 Related Work

Quantum transpilers and error mitigation. We can categorize quantum circuit transpilation techniques as (1) qubit mapping and routing [41, 49, 50, 60, 63, 80, 85, 87, 93, 96, 99], (2) instruction/pulse scheduling [17, 27, 51, 77, 81, 90, 98] and (3) gate optimization/decomposition [16, 43, 60, 64, 77, 94]. Finally, there is work on post-execution processing, readout improvement, and error correction [12, 15, 18, 45, 46, 61, 62, 84, 86]. These proposals are orthogonal to our work and can be integrated into QVM. This is especially the case for measurement error mitigation, which can help to improve the fidelity of the mid-circuit measurements during execution [79, 95].

Circuit cutting and knitting. Circuit cutting & knitting is the process of breaking down a large quantum circuit into smaller sub-circuits that can be executed separately, then synthesizing the results to obtain the result of the original circuit. Circuit cutting can be divided into gate virtualization (§ 2.3) and *wire cutting* [13, 14, 65, 83, 91]. Wire cutting has been primarily explored in CutQC [83], building on the work of Peng et al. [65], which provides a method for cutting the time-evolution of larger circuits to decompose them into smaller subcircuits. We propose a generic system for gate virtualization that not only enables circuit cutting but also reduces qubit dependency, improving fidelity and facilitating qubit recycling. Ideally, these methods should be combined in the future to further advance circuit cutting techniques.

Qubit reuse / recycling. Qubit reuse can be classified into two categories, namely ancilla reuse using *uncomputation* [9] and reuse through *dynamic circuits* [3, 32]. Work such as [10, 21, 59] utilize uncomputation to reclaim ancilla qubits. In contrast, work such as [20, 31, 58, 74] exploit the newly supported dynamic circuits with mid-circuit measurements and mid-circuit reset operations to reuse qubits. Jiang’s work [35] formalizes the qubit recycling problem using qubit-dependency graphs and develops a heuristic solver and a verified qubit recycler. However, applying these techniques on densely connected circuits can be impractical due to the large number of qubit dependencies [31, 88]. By first applying QVM’s DR pass (§ 4.4), qubit reuse can be practically applied with enhanced efficiency. Therefore, our work highlights a promising approach to integrating gate virtualization and qubit recycling, enabling circuit optimizations that neither technique could achieve alone.

Application-specific optimizations. Application-specific circuit optimizations go beyond generic strategies and target the unique characteristics of a particular algorithm or circuit structure in order to improve fidelity [5, 6, 25, 26, 39, 42, 82]. Our work tries to build a generic and extensible framework to incorporate different application-specific optimizations.

Quantum cloud computing. This area addresses quantum circuit multi-programming [19, 44, 56, 57], quantum resource management/scheduling [72, 73, 92], and quantum serverless [23, 54]. Our work is complimentary to these proposals, QVM proposes a scalable infrastructure for supporting gate virtualization optimizations, which can be incorporated by quantum cloud environments.

8 Conclusion

We introduce the Quantum Gate Virtualization Machine (QVM), a generic system for the scalable, high-fidelity execution of large circuits on noisy and small QPUs by leveraging gate virtualization to enable the execution of circuits that neither QPUs nor classical computers could run alone. QVM extends the quantum circuit abstraction with the *virtual circuit IR*, which forms the foundation for the QVM Compiler—a modular compiler infrastructure for implementing a series of optimization passes to generate smaller, optimized fragments. These fragments are virtualized and executed using our QVM Runtime—a distributed and scalable system to execute and post-process the instantiated circuit fragments in a highly parallel manner on a distributed set of QPUs. Our evaluation on IBM’s 7- and 27-qubit QPUs of QVM demonstrates practical scaling of circuits with sizes up to double the QPU capacity while significantly improving fidelity.

Artifact. The artifact is publicly available at github.com/TUM-DSE/qvm [89].

Acknowledgements

We thank Karl Jansen and Stefan Kühn for supporting this work by providing access to IBM quantum resources. We thank Martin Ruefenacht for his valuable contributions during his employment at the Leibniz Supercomputing Center. We also thank Ahmed Darwish and Francisco Romão for their contributions to this work. This work was supported by the Bavarian State Ministry of Science and the Arts with funds from the Hightech Agenda Bayern Plus, as part of the Munich Quantum Valley (MQV) initiative (6090181).

References

- [1] [n. d.]. Qiskit Transpiler. <https://qiskit.org/documentation/apidoc/transpiler.html>. Accessed: 2022-06-09.
- [2] [n. d.]. Quantum Computer Datasheet. <https://quantumai.google/hardware/datasheet/weber.pdf>. Accessed: 2023-07-17.
- [3] [n. d.]. The IBM Quantum Development Roadmap. <https://www.ibm.com/quantum/roadmap>. Accessed: 2022-06-02.
- [4] Google Quantum AI. 2024. Willow Quantum Processor Specification Sheet. <https://quantumai.google/static/site-assets/downloads/willow-spec-sheet.pdf>. Accessed: 2025-03-10.
- [5] Mahabubul Alam, Abdullah Ash-Saki, and Swaroop Ghosh. 2020. Circuit Compilation Methodologies for Quantum Approximate Optimization Algorithm. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 215–228. doi:10.1109/MICRO50266.2020.00029
- [6] Ramin Ayanzadeh, Narges Alavisamani, Poulami Das, and Moinuddin Qureshi. 2023. FrozenQubits: Boosting Fidelity of QAOA by Skipping Hotspot Nodes. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Vancouver, BC, Canada) (ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 311–324. doi:10.1145/3575693.3575741
- [7] Marvin Bechtold, Johanna Barzen, Frank Leymann, Alexander Mandl, Julian Obst, Felix Truger, and Benjamin Weder. 2023. Investigating the effect of circuit cutting in QAOA for the MaxCut problem on NISQ devices. *arXiv preprint arXiv:2302.01792* (2023).
- [8] Luciano Bello, Agata M. Brańczyk, Sergey Bravyi, Almudena Carrera Vazquez, Andrew Eddins, Daniel J. Egger, Bryce Fuller, Julien Gacon, James R. Garrison, Jennifer R. Glick, Tanvi P. Gujarati, Ikko Hamamura, Areeq I. Hasan, Takashi Imamichi, Caleb Johnson, Ieva Liepuoniute, Owen Lockwood, Mario Motta, C. D. Pemmaraju, Pedro Rivero, Max Rossmannek, Travis L. Scholten, Seetharami Seelam, Iskandar Sitdikov, Dharmashankar Subramanian, Wei Tang, and Stefan Woerner. 2023. Circuit Knitting Toolbox. <https://github.com/Qiskit-Extensions/circuit-knitting-toolbox>. doi:10.5281/zenodo.7987997
- [9] C. H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (Nov 1973), 525–532. doi:10.1147/rd.176.0525
- [10] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (London, UK) (PLDI 2020)*. 286–300. doi:10.1145/3385412.3386007
- [11] Sebastian Brandhofer, Ilia Polian, and Kevin Krsulich. 2023. Optimal Partitioning of Quantum Circuits using Gate Cuts and Wire Cuts. *arXiv preprint arXiv:2308.09567* (2023).
- [12] Sergey Bravyi, Sarah Sheldon, Abhinav Kandala, David C. McKay, and Jay M. Gambetta. 2021. Mitigating measurement errors in multiqubit experiments. *Phys. Rev. A* 103 (Apr 2021), 042605. Issue 4. doi:10.1103/PhysRevA.103.042605
- [13] Sergey Bravyi, Graeme Smith, and John A. Smolin. 2016. Trading Classical and Quantum Computational Resources. *Phys. Rev. X* 6 (Jun 2016), 021043. Issue 2. doi:10.1103/PhysRevX.6.021043
- [14] Lukas Brenner, Christophe Piveteau, and David Sutter. 2023. Optimal wire cutting with classical communication. *arXiv preprint arXiv:2302.03366* (2023).
- [15] Siddharth Dangwal, Gokul Subramanian Ravi, Poulami Das, Kaitlin N Smith, Jonathan Mark Baker, and Frederic T Chong. 2023. Varsaw: Application-tailored measurement error mitigation for variational quantum algorithms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 362–377.
- [16] Poulami Das, Eric Kessler, and Yunong Shi. 2023. The Imitation Game: Leveraging CopyCats for Robust Native Gate Selection in NISQ Programs. In *International Symposium on High-Performance Computer Architecture (HPCA)*. doi:10.1109/HPCA56546.2023.10071025
- [17] Poulami Das, Swamit Tannu, Siddharth Dangwal, and Moinuddin Qureshi. 2021. ADAPT: Mitigating Idling Errors in Qubits via Adaptive Dynamical Decoupling. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 937–949. doi:10.1145/3466752.3480044
- [18] Poulami Das, Swamit Tannu, and Moinuddin Qureshi. 2021. JigSaw: Boosting Fidelity of NISQ Programs via Measurement Subsetting. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (Virtual Event, Greece) (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 937–949. doi:10.1145/3466752.3480044
- [19] Poulami Das, Swamit S. Tannu, Prashant J. Nair, and Moinuddin Qureshi. 2019. A Case for Multi-Programming Quantum Computers. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (Columbus, OH, USA) (MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 291–303. doi:10.1145/3352460.3358287
- [20] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2022. Qubit-reuse compilation with mid-circuit measurement and reset. *arXiv preprint arXiv:2210.08039* (2022).
- [21] Yongshan Ding, Xin-Chuan Wu, Adam Holmes, Ash Wiseth, Diana Franklin, Margaret Martonosi, and Frederic T. Chong. 2020. SQUARE: Strategic Quantum Ancilla Reuse for Modular Quantum Programs via Cost-Effective Uncomputation. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 570–583. doi:10.1109/ISCA45697.

- 2020.00054
- [22] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
 - [23] Jose Garcia-Alonso, Javier Rojo, David Valencia, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2022. Quantum Software as a Service Through a Quantum API Gateway. *IEEE Internet Computing* 26, 1 (Jan 2022), 34–41. doi:10.1109/MIC.2021.3132688
 - [24] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. 2012. *Answer Set Solving in Practice*. Morgan & Claypool Publishers.
 - [25] Pranav Gokhale, Olivia Angiuli, Yongshan Ding, Kaiwen Gui, Teague Tomesh, Martin Suchara, Margaret Martonosi, and Frederic T Chong. 2019. Minimizing state preparations in variational quantum eigensolver by partitioning into commuting families. *arXiv preprint arXiv:1907.13623* (2019).
 - [26] Pranav Gokhale, Yongshan Ding, Thomas Propson, Christopher Winkler, Nelson Leung, Yunong Shi, David I. Schuster, Henry Hoffmann, and Frederic T. Chong. 2019. Partial Compilation of Variational Algorithms for Noisy Intermediate-Scale Quantum Machines. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (*MICRO '52*). Association for Computing Machinery, New York, NY, USA, 266–278. doi:10.1145/3352460.3358313
 - [27] Pranav Gokhale, Ali Javadi-Abhari, Nathan Earnest, Yunong Shi, and Frederic T. Chong. 2020. Optimized Quantum Compilation for Near-Term Algorithms with OpenPulse. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 186–200. doi:10.1109/MICRO50266.2020.00027
 - [28] Guillermo González-García, Rahul Trivedi, and J Ignacio Cirac. 2022. Error propagation in nisy devices for solving classical optimization problems. *PRX Quantum* 3, 4 (2022), 040326.
 - [29] Riddhi Swaroop Gupta, Ewout van den Berg, Maika Takita, Kristan Temme, and Abhinav Kandala. 2024. Probabilistic error cancellation for dynamic quantum circuits. *Bulletin of the American Physical Society* (2024).
 - [30] Ernst Hellinger. 1909. Neue begründung der theorie quadratischer formen von unendlichvielen veränderlichen. *Journal für die reine und angewandte Mathematik* 1909, 136 (1909), 210–271.
 - [31] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S Bishop, John Lapere, Ali Javadi-Abhari, and Eddy Z Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 59–71.
 - [32] IBM. n.d.. Getting started with dynamic circuits. <https://quantum-computing.ibm.com/services/programs/docs/runtime/manage/system/circuits/Getting-started-with-Dynamic-Circuits>. Accessed: 2022-06-02.
 - [33] IBM Quantum. 2023. IBM Quantum — quantum-computing.ibm.com. <https://quantum-computing.ibm.com/>. Accessed: 2023-07-28.
 - [34] QuEra Computing Inc. 2023. Harvard University, MIT and QuEra Demonstrate Historic 99.5% Two-Qubit Gate Fidelity on 60 Neutral Atom Qubits. <https://www.quera.com/press-releases/harvard-university-mit-and-quera-demonstrate-historic-99-5-two-qubit-gate-fidelity-on-60-neutral-atom-qubits> Accessed: 2025-03-09.
 - [35] Hanru Jiang. 2024. Qubit recycling revisited. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 1264–1287.
 - [36] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. 2017. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *nature* 549, 7671 (2017), 242–246.
 - [37] Brian W Kernighan and Shen Lin. 1970. An efficient heuristic procedure for partitioning graphs. *The Bell system technical journal* 49, 2 (1970), 291–307.
 - [38] Ilyas Khan and Jenni Strabley. 2024. Quantinuum extends its significant lead in quantum computing, achieving historic milestones for hardware fidelity and Quantum Volume. <https://www.quantinuum.com/blog/quantinuum-extends-its-significant-lead-in-quantum-computing-achieving-historic-milestones-for-hardware-fidelity-and-quantum-volume> Accessed: 2025-03-09.
 - [39] Lingling Lao and Dan E. Browne. 2022. 2QAN: A Quantum Compiler for 2-Local Qubit Hamiltonian Simulation Algorithms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) (*ISCA '22*). Association for Computing Machinery, New York, NY, USA, 351–365. doi:10.1145/3470496.3527394
 - [40] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Transactions on Quantum Computing* 4, 2, Article 10 (feb 2023), 26 pages. doi:10.1145/3550488
 - [41] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (*ASPLOS '19*). Association for Computing Machinery, New York, NY, USA, 1001–1014. doi:10.1145/3297858.3304023

- [42] Gushu Li, Anbang Wu, Yunong Shi, Ali Javadi-Abhari, Yufei Ding, and Yuan Xie. 2022. Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 554–569. doi:10.1145/3503222.3507715
- [43] Andrew Litteken, Lennart Maximilian Seifert, Jason D. Chadwick, Natalia Nottingham, Tanay Roy, Ziqian Li, David Schuster, Frederic T. Chong, and Jonathan M. Baker. 2023. Dancing the Quantum Waltz: Compiling Three-Qubit Gates on Four Level Architectures. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 71, 14 pages. doi:10.1145/3579371.3589106
- [44] Lei Liu and Xinglei Dou. 2021. QuCloud: A New Qubit Mapping Mechanism for Multi-programming Quantum Computing in Cloud Environment. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 167–178. doi:10.1109/HPCA51647.2021.00024
- [45] Filip B Maciejewski, Zoltán Zimborás, and Michał Oszmaniec. 2020. Mitigation of readout noise in near-term quantum devices by classical post-processing based on detector tomography. *Quantum* 4 (2020), 257.
- [46] Satvik Maurya, Chaithanya Naik Mude, William D. Oliver, Benjamin Lienhard, and Swamit Tannu. 2023. Scaling Qubit Readout with Hardware Efficient Machine Learning Architectures. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA '23). Association for Computing Machinery, New York, NY, USA, Article 7, 13 pages. doi:10.1145/3579371.3589042
- [47] Kosuke Mitarai and Keisuke Fujii. 2021. Constructing a virtual two-qubit gate by sampling single-qubit operations. *New Journal of Physics* 23, 2 (2021), 023021.
- [48] Kosuke Mitarai and Keisuke Fujii. 2021. Overhead for simulating a non-local channel with local channels by quasiprobability sampling. *Quantum* 5 (2021), 388.
- [49] Abtin Molavi, Amanda Xu, Martin Diges, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2022. Qubit Mapping and Routing via MaxSAT. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1078–1091. doi:10.1109/MICRO56248.2022.00077
- [50] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 1015–1029. doi:10.1145/3297858.3304075
- [51] Prakash Murali, David C. McKay, Margaret Martonosi, and Ali Javadi-Abhari. 2020. Software Mitigation of Crosstalk on Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '20). 1001–1016.
- [52] Paul Nation, Matthew Treinish, and Clemens Pessel. 2022. mapomatic. <https://github.com/Qiskit-Partners/mapomatic>.
- [53] Paul D Nation and Matthew Treinish. 2023. Suppressing quantum circuit errors due to system variability. *PRX Quantum* 4, 1 (2023), 010327.
- [54] Hoa T Nguyen, Muhammad Usman, and Rajkumar Buyya. 2022. Qfaas: A serverless function-as-a-service framework for quantum computing. *arXiv preprint arXiv:2205.14845* (2022).
- [55] Michael A Nielsen and Isaac L Chuang. 2010. *Quantum computation and quantum information*. Cambridge university press.
- [56] Siyuan Niu and Aida Todri-Sanial. 2023. Enabling Multi-programming Mechanism for Quantum Computing in the NISQ Era. *Quantum* 7 (feb 2023), 925. doi:10.22331/q-2023-02-16-925
- [57] Yasuhiro Ohkura, Takahiko Satoh, and Rodney Van Meter. 2022. Simultaneous Execution of Quantum Circuits on Current and Near-Future NISQ Systems. *IEEE Transactions on Quantum Engineering* 3 (2022), 1–10. doi:10.1109/TQE.2022.3164716
- [58] Alexandru Paler, Robert Wille, and Simon J. Devitt. 2016. Wire recycling for quantum circuit optimization. *Phys. Rev. A* 94 (Oct 2016), 042337. Issue 4. doi:10.1103/PhysRevA.94.042337
- [59] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 222–236. doi:10.1145/3453483.3454040
- [60] Tirthak Patel, Daniel Silver, and Devesh Tiwari. 2022. Geyser: A Compilation Framework for Quantum Computing with Neutral Atoms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) (ISCA '22). Association for Computing Machinery, New York, NY, USA, 383–395. doi:10.1145/3470496.3527428
- [61] Tirthak Patel and Devesh Tiwari. 2020. DisQ: A Novel Quantum Output State Classification Method on IBM Quantum Computers Using Openpulse. In *Proceedings of the 39th International Conference on Computer-Aided Design* (Virtual Event, USA) (ICCAD '20). Association for Computing Machinery, New York, NY, USA, Article 139, 9 pages. doi:10.

- 1145/3400302.3415619
- [62] Tirthak Patel and Devesh Tiwari. 2020. VERITAS: Accurately Estimating the Correct Output on Noisy Intermediate-Scale Quantum Computers. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16. doi:10.1109/SC41405.2020.00019
 - [63] Tirthak Patel and Devesh Tiwari. 2021. Graft: Reverse Your Quantum Circuit and Know the Correct Program Output. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 443–455. doi:10.1145/3445814.3446743
 - [64] Tirthak Patel, Ed Younis, Costin Iancu, Wibe de Jong, and Devesh Tiwari. 2022. QUEST: Systematically Approximating Quantum Circuits for Higher Output Fidelity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Lausanne, Switzerland) (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 514–528. doi:10.1145/3503222.3507739
 - [65] Tianyi Peng, Aram W. Harrow, Maris Ozols, and Xiaodi Wu. 2020. Simulating Large Quantum Circuits on a Small Quantum Computer. *Phys. Rev. Lett.* 125 (Oct 2020), 150504. Issue 15. doi:10.1103/PhysRevLett.125.150504
 - [66] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5, 1 (2014), 4213.
 - [67] Christophe Piveteau and David Sutter. 2022. Circuit knitting with classical communication. *arXiv preprint arXiv:2205.00016* (2022).
 - [68] Potassco. 2023. Clingo: A grounder and solver for logic programs. <https://github.com/potassco/clingo>. Accessed: 2023-07-17.
 - [69] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (Aug. 2018), 79. doi:10.22331/q-2018-08-06-79
 - [70] Qiskit contributors. 2023. Qiskit: An Open-source Framework for Quantum Computing. doi:10.5281/zenodo.2573505
 - [71] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2022. MQT Bench: Benchmarking software and design automation tools for quantum computing. *arXiv preprint arXiv:2204.13719* (2022).
 - [72] Gokul Subramanian Ravi, Kaitlin N. Smith, Pranav Gokhale, and Frederic T. Chong. 2021. Quantum Computing in the Cloud: Analyzing job and machine characteristics. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. 39–50. doi:10.1109/IISWC53511.2021.00015
 - [73] Gokul Subramanian Ravi, Kaitlin N. Smith, Prakash Murali, and Frederic T. Chong. 2021. Adaptive job and resource management for the growing quantum cloud. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. 301–312. doi:10.1109/QCE52317.2021.00047
 - [74] Movahhed Sadeghi, Soheil Khadirsharbiyani, and Mahmut Taylan Kandemir. 2022. Quantum Circuit Resizing. *arXiv preprint arXiv:2301.00720* (2022).
 - [75] Zain H Saleem, Teague Tomesh, Michael A Perlin, Pranav Gokhale, and Martin Suchara. 2021. Quantum divide and conquer for combinatorial optimization and distributed computing. *arXiv preprint arXiv:2107.07532* 10 (2021).
 - [76] Lukas Schmitt, Christophe Piveteau, and David Sutter. 2025. Cutting circuits with multiple two-qubit unitaries. *Quantum* 9 (2025), 1634.
 - [77] Yunong Shi, Nelson Leung, Pranav Gokhale, Zane Rossi, David I. Schuster, Henry Hoffmann, and Frederic T. Chong. 2019. Optimized Compilation of Aggregated Instructions for Realistic Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*.
 - [78] Peter W. Shor. 1999. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Rev.* 41, 2 (1999), 303–332. doi:10.1137/S0036144598347011 arXiv:<https://doi.org/10.1137/S0036144598347011>
 - [79] Akhil Pratap Singh, Kosuke Mitarai, Yasunari Suzuki, Kentaro Heya, Yutaka Tabuchi, Keisuke Fujii, and Yasunobu Nakamura. 2023. Experimental demonstration of a high-fidelity virtual two-qubit gate. *arXiv preprint arXiv:2307.03232* (2023).
 - [80] Marcos Yukio Siraichi, Vinicius Fernandes dos Santos, Caroline Collange, and Fernando Magno Quintao Pereira. 2018. Qubit Allocation. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization (Vienna, Austria) (CGO 2018)*. Association for Computing Machinery, New York, NY, USA, 113–125. doi:10.1145/3168822
 - [81] Kaitlin N. Smith, Gokul Subramanian Ravi, Prakash Murali, Jonathan M. Baker, Nathan Earnest, Ali Javadi-Cabbari, and Frederic T. Chong. 2022. TimeStitch: Exploiting Slack to Mitigate Decoherence in Quantum Circuits. *ACM Transactions on Quantum Computing* 4, 1, Article 8 (oct 2022), 27 pages. doi:10.1145/3548778
 - [82] Samuel Stein, Nathan Wiebe, Yufei Ding, Peng Bo, Karol Kowalski, Nathan Baker, James Ang, and Ang Li. 2022. EQC: Ensembled Quantum Computing for Variational Quantum Algorithms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (New York, New York) (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 59–71. doi:10.1145/3470496.3527434

- [83] Wei Tang, Teague Tomesh, Martin Suchara, Jeffrey Larson, and Margaret Martonosi. 2021. CutQC: Using Small Quantum Computers for Large Quantum Circuit Evaluations. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 473–486. doi:10.1145/3445814.3446758
- [84] Swamit Tannu, Poulami Das, Ramin Ayanzadeh, and Moinuddin Qureshi. 2022. HAMMER: Boosting Fidelity of Noisy Quantum Circuits by Exploiting Hamming Behavior of Erroneous Outcomes. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '22). Association for Computing Machinery, New York, NY, USA, 529–540. doi:10.1145/3503222.3507703
- [85] Swamit S. Tannu and Moinuddin Qureshi. 2019. Ensemble of Diverse Mappings: Improving Reliability of Quantum Computers by Orchestrating Dissimilar Mistakes. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 253–265. doi:10.1145/3352460.3358257
- [86] Swamit S. Tannu and Moinuddin K. Qureshi. 2019. Mitigating Measurement Errors in Quantum Computers by Exploiting State-Dependent Bias. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 279–290. doi:10.1145/3352460.3358265
- [87] Swamit S. Tannu and Moinuddin K. Qureshi. 2019. Not All Qubits Are Created Equal: A Case for Variability-Aware Policies for NISQ-Era Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 987–999. doi:10.1145/3297858.3304007
- [88] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R Martonosi, and Frederic T Chong. 2022. Supermarq: A scalable quantum benchmark suite. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 587–603.
- [89] Nathaniel Tornow and Emmanouil Giortamis. 2025. QVM: Quantum Gate Virtualization Machine. doi:10.5281/zenodo.15040054
- [90] Vinay Tripathi, Huo Chen, Mostafa Khezri, Ka-Wa Yip, E.M. Levenson-Falk, and Daniel A. Lidar. 2022. Suppression of Crosstalk in Superconducting Qubits Using Dynamical Decoupling. *Phys. Rev. Appl.* 18 (Aug 2022), 024068. Issue 2. doi:10.1103/PhysRevApplied.18.024068
- [91] Christian Ufrecht, Maniraman Periyasamy, Sebastian Rietsch, Daniel D Scherer, Axel Plinge, and Christopher Mutschler. 2023. Cutting multi-control quantum gates with ZX calculus. *arXiv preprint arXiv:2302.00387* (2023).
- [92] Benjamin Weder, Johanna Barzen, Frank Leymann, and Marie Salm. 2021. Automated Quantum Hardware Selection for Quantum Workflows. *Electronics* 10, 8 (2021). doi:10.3390/electronics10080984
- [93] Robert Wille, Lukas Burgholzer, and Alwin Zulehner. 2019. Mapping Quantum Circuits to IBM QX Architectures Using the Minimal Number of SWAP and H Operations. In *Proceedings of the 56th Annual Design Automation Conference 2019* (Las Vegas, NV, USA) (DAC '19). Association for Computing Machinery, New York, NY, USA, Article 142, 6 pages. doi:10.1145/3316781.3317859
- [94] Amanda Xu, Abtin Molavi, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2023. Synthesizing Quantum-Circuit Optimizers. *Proc. ACM Program. Lang.* 7, PLDI, Article 140 (jun 2023), 25 pages. doi:10.1145/3591254
- [95] Takahiro Yamamoto and Ryutaro Ohira. 2022. Error suppression by a virtual two-qubit gate. *arXiv preprint arXiv:2212.05493* (2022).
- [96] Chi Zhang, Ari B. Hayes, Longfei Qiu, Yuwei Jin, Yanhao Chen, and Eddy Z. Zhang. 2021. Time-Optimal Qubit Mapping. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLOS '21). Association for Computing Machinery, New York, NY, USA, 360–374. doi:10.1145/3445814.3446706
- [97] Jun Zhang, Jiri Vala, Shankar Sastry, and K Birgitta Whaley. 2003. Geometric theory of nonlocal two-qubit operations. *Physical Review A* 67, 4 (2003), 042313.
- [98] Alexander Zlokapa and Alexandru Gheorghiu. 2020. A deep learning model for noise prediction on near-term quantum devices. *arXiv preprint arXiv:2005.10811* (2020).
- [99] Alwin Zulehner, Alexandru Paler, and Robert Wille. 2019. An Efficient Methodology for Mapping Quantum Circuits to the IBM QX Architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 7 (July 2019), 1226–1236. doi:10.1109/TCAD.2018.2846658

Received 2024-11-14; accepted 2025-03-06